TD_TP N°1

Exercice 1 :

Soit P un pointeur qui pointe sur un tableau A :

```
int A[] = {12, 23, 34, 45, 56, 67, 78, 89, 90};
int *P;
P = A;
```

Quelles valeurs ou adresses fournissent ces expressions :

a) `*P+2` b) `*(P+2)` c) `&P+1` d) `&A[4]-3` e) `A+3`

f) `&A[7]-P` g) `P+(*P-10)` h) `*(P+*(P+8)-A[7])`

Exercice 2 :

Ecrire un programme qui lit 10 phrases d'une longueur maximale de 200 caractères au clavier et qui les mémorise dans un tableau de pointeurs sur char en réservant dynamiquement l'emplacement en mémoire pour les chaines. Ensuite, l'ordre des phrases est inversé en modifiant les pointeurs et le tableau résultant est affiché.

Exercice 3:

Ecrire un programme qui lit un entier X et un tableau A du type int au clavier et élimine toutes les occurrences de X dans A en tassant les éléments restants. Le programme utilisera les pointeurs P1 et P2 pour parcourir le tableau.

Exercice 4 :

Ecrire un programme qui lit deux tableaux d'entiers A et B et leurs dimensions N et M au clavier et qui ajoute les éléments de B à la fin de A. utiliser deux pointeurs PA et PB pour le transfert et afficher le tableau résultant.

Exercice 5:

Ecrire de deux façons différentes, un programme qui vérifie sans utiliser une fonction de <string>, si une chaine est palindrome :

a) En utilisant uniquement le formalisme tableau.
b) En utilisant des pointeurs au lieu des indices numériques.

***Rappel** : un mot palindrome est un mot qui reste le même qu'on le lise de gauche à droite ou de droite à gauche.*
***Exemples** : RADAR, 23432*

Exercice 6:

Ecrire un programme qui demande un nom et affiche le nom inversé. On utilisera des pointeurs de caractères à la place du tableau de caractères. Le programme allouera dynamiquement une chaîne de 30 caractères, et emploiera deux pointeurs de caractères pour réaliser l'inversion. Le premier pointeur progressera du début vers la fin de la chaîne, et le second de la fin vers le début. Le programme doit inverser les caractères tant que les deux pointeurs ne se sont pas rencontrés.

Exercice 7:

Écrire une fonction qui calcule la somme et le produit des éléments d'un tableau passé en paramètre. Écrire le programme principal qui initialise le tableau par saisie ; calcule et affiche la somme et le produit des éléments.

Exercice 8:

– Définir la structure permettant de représenter un nombre complexe. Définir les fonctions complexes suivantes (passages par valeur) : `double imag(Complex z), double real (Complex z), Complex mul(Complex z1, Complex z2), double abs(Complex z),` et Réécrire ces fonctions en utilisant le passage par adresses.

Exercice 9:

Une menuiserie industrielle gère un stock de panneaux de bois. Chaque panneau possède une largeur, une longueur et une épaisseur en millimètres, ainsi que le type de bois qui peut être Pin (code 0), chêne (code 1) ou hêtre (code 2).

- Définir une structure `Panneau` contenant toutes les informations relatives à un panneau de bois.
- Écrire des fonctions de saisie et d'affichage d'un panneau de bois.
- Écrire une fonction qui calcule le volume en mètres cube d'un panneau.

Exercice 10:

Un grossiste en composants électroniques vend quatre types de produits :

a) Des cartes mères (code 1) ; Des processeurs (code 2) ; Des barètes mémoire (code 3) ;
b) Des cartes graphiques (code 4).
c) Chaque produit possède une référence (qui est un nombre entier), un prix en euros et des quantités disponibles.

a) Définir une structure `Produit` qui code un produit.
b) Écrire une fonction de saisie et d'affichage des données d'un produit.
c) Écrire une fonction qui permet à un utilisateur de saisir une commande d'un produit. L'utilisateur saisit les quantités commandées et les données du produit. L'ordinateur affiche toutes les données de la commande, y compris le prix.

Exercice 11:

Considérons la structure `TypeTableau` suivante qui contient l'adresse d'un tableau et le nombre d'éléments du tableau.

```
typedef struct{
            int nb_elem;       /* nombre d'éléments */
            int *tab;          /* tableau */
}TypeTableau;
```

1) Écrire les fonctions suivantes:

- `TypeTableau CreationTableau(int n);` qui crée un tableau de n éléments.
- `void DestructionTableau(TypeTableau T);` qui libère la mémoire occupée par un tableau.
- `void SimpleLectureTableau(TypeTableau T);` qui lit les éléments d'un tableau au clavier. On supposera dans cette fonction que le tableau a déjà été alloué précédemment.
- `void Affichage(TypeTableau T);` qui affiche le contenu d'un tableau.
- `TypeTableau DoubleTableau(TypeTableau T);` qui crée un nouveau tableau de même taille que T mais dont les éléments sont le double des éléments de T.

2) Écrire un programme principal qui saisit un tableau au clavier, calcule le double de chaque élément du tableau et affiche les résultats.

Exercice 12:

Soient deux modèles de structure `date` et `employe` déclarés ainsi :

```
struct date {
       int jour; char mois[10]; int annee;
};
struct employe {
          char nom[15], prenom[15]
          struct date anniversire, date_embauche;
};
```

Ecrire un programme qui lit des informations pour 4 employés. Et Affiche les employés sous la forme suivante :

```
Nom : Maghribi
Prenom : Mohammed
Date de naissance : 21 Janvier 1992
Date d'embauche :  05 Octobre 2010
```

Exercice 13:

Soient le modèle de structure `etudiant` déclarés ainsi :

```
struct etudiant{
    char nom[15] ,prenom[15];
    int CNE;
    float notes[4] , moyenne;
};
```

Ecrire un programme qui :

- Déclare un tableau d'étudiants T.
- Lit les informations de 5 étudiants.
- Cherche l'étudiant ayant la plus grande moyenne, puis affiche ses informations.
- Trie le tableau de structures T en ordre décroissant selon la moyenne.

Exercice 14:

Définir une structure `NombreRationnel` permettant de coder un nombre rationnel, avec numérateur et dénominateur. On écrira des fonctions de saisie, d'affichage, de multiplication et d'addition de deux rationnels. Pour l'addition, pour simplifier, on ne cherchera pas nécessairement le plus petit dénominateur commun.